Regression Optimization Using Numerical Methods

Mini Project 1

Saroar Jahan Shuba

03/10/2026

Objectives

The purpose of this mini project is to study how numerical optimization methods estimate regression parameters under different modeling assumptions. The project combines three layers of analysis. The first layer is the regression model itself: a simple linear regression (SLR) model, which provides a convex optimization benchmark, and a cubic polynomial fitting (CPF) model, which introduces a more complex optimization setting. The second layer is the optimization algorithm: Batch Gradient Descent (BGD) and Stochastic Gradient Descent (SGD), where SGD is implemented both with replacement and without replacement. The third layer is the objective function: Maximum Likelihood Estimation (MLE) and Bayesian inference through the log-posterior.

The main goals of the project are the following:

1. generate synthetic regression datasets with Gaussian noise so that the true parameters are known;
2. derive the cost functions and gradients for SLR under both MLE and Bayesian formulations;
3. derive the MLE cost function and gradients for cubic polynomial fitting;
4. implement BGD and SGD in Python and compare their convergence behaviors;
5. evaluate how different optimization choices affect parameter estimates, final cost values, and fitted curves; and
6. interpret the behavior of the algorithms using plots, parameter summaries, and optimization trajectories.

In this project, all regression models, objective functions, and optimization algorithms were implemented in Python from scratch using NumPy, and the results were analyzed through cost histories, parameter trajectories, fitted curves, and summary tables.

Data Generation

1. Data generation strategy

Both datasets in this project were generated synthetically so that the true model parameters are known in advance. This is useful because the performance of each optimization method can then be evaluated against a known ground truth. In both models, the response variable was generated by adding Gaussian noise to the deterministic regression function.

The choice of synthetic data is also consistent with the purpose of the project. Since the main focus is optimization rather than domain-specific data collection, simulated data provides full control over the regression structure, the signal-to-noise ratio, and the sample size.

2. Simple linear regression data

For the simple linear regression experiment, the data was generated from

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

The true parameters used in the code were

$$\beta_0 = 2, \qquad \beta_1 = 3, \qquad \sigma = 1.$$

These values were chosen so that the linear relationship would be clearly visible in the generated data while still allowing the Gaussian noise to create noticeable variation around the true line.

The explanatory variable was drawn from a uniform distribution on the interval $[-2, 2]$, and the sample size was set to $n = 120$. In the code, the data were stored using one-dimensional NumPy arrays:

Listing 1: SLR data structure used in the code.

```
x_slr.shape = (120,)
y_slr.shape = (120,)
```

This structure is important because the cost functions and gradients were implemented in vectorized form. A one-dimensional array for x and y keeps the expressions simple while still allowing efficient numerical computation.

Figure 1 shows the generated data together with the true regression line. The plot confirms a clear positive linear relationship with Gaussian variability around the underlying line.

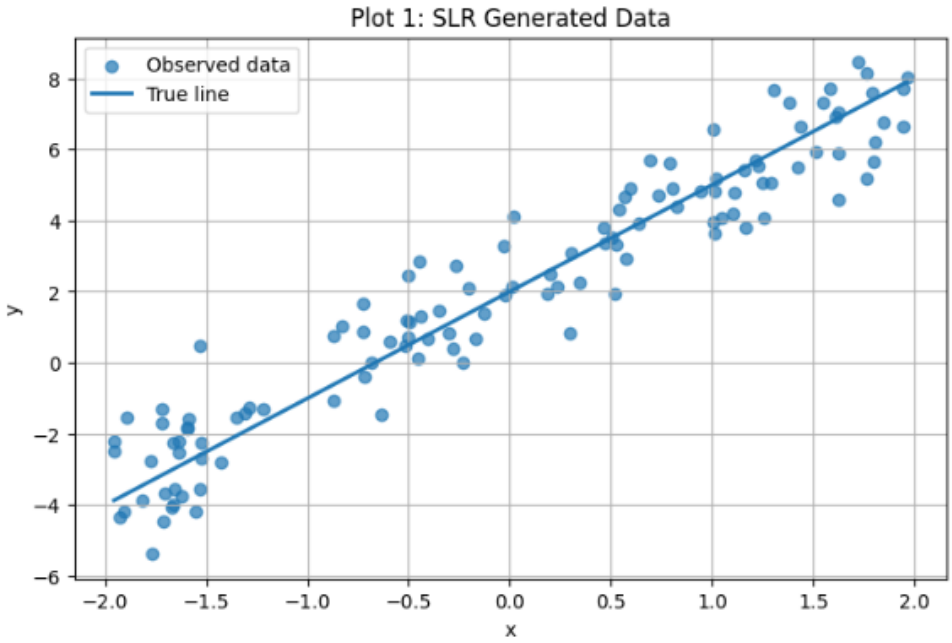

Figure 1: Synthetic dataset generated for the simple linear regression experiment.

3. Cubic polynomial data

For the cubic polynomial fitting experiment, the response was generated from

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

The true parameters were chosen as

$$\beta_0 = 1, \qquad \beta_1 = -2, \qquad \beta_2 = 0.5, \qquad \beta_3 = 1.2, \qquad \sigma = 2.$$

These coefficients were selected so that the generated response would display a clearly nonlinear pattern, making it possible to distinguish the cubic regression setting from the simple linear case.

The raw predictor values were again sampled from a uniform distribution on $[-2, 2]$.

However, before fitting the model, the raw predictor was standardized in the code:

$$z = \frac{x - \bar{x}}{s_x}.$$

This choice was necessary for numerical stability. Since polynomial terms such as x^2 and x^3 can grow rapidly in magnitude, standardizing the predictor reduces scale imbalance among the features and helps gradient descent converge more smoothly.

The final arrays in the code were stored as

Listing 2: CPF data structure used in the code.

```
x_cpf.shape = (150,)
y_cpf.shape = (150,)
```

Thus, the cubic model was built from a one-dimensional standardized predictor and its polynomial powers inside the prediction function. Figure 2 shows the generated cubic dataset and the true cubic curve. The nonlinear structure is clearly visible in the plot.

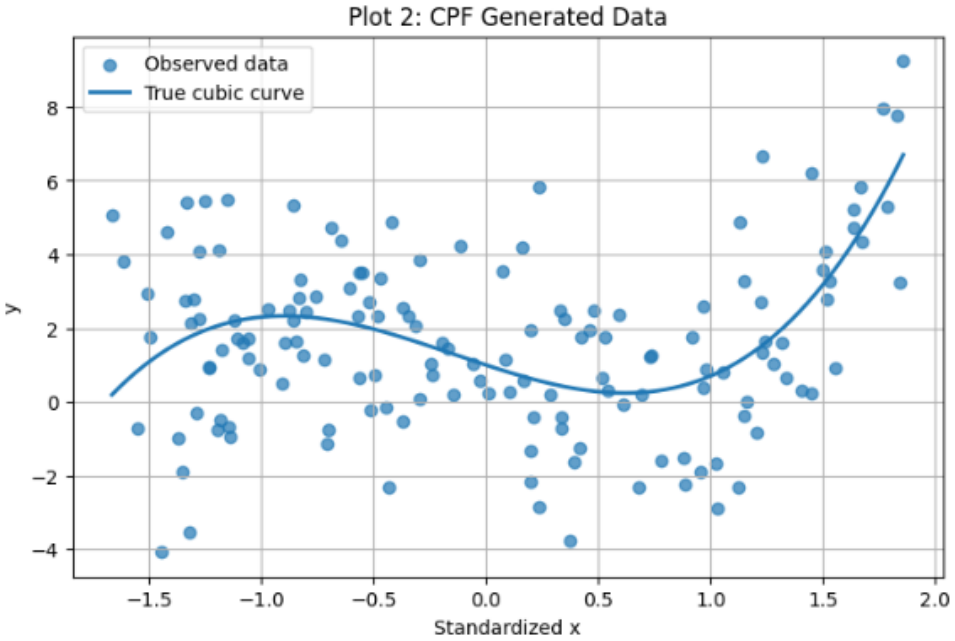

Figure 2: Synthetic dataset generated for cubic polynomial regression.

4. Summary of Data Structures Used in the Code

The datasets used in this project were stored using NumPy arrays. These arrays contain the predictor variables and response variables used for both regression models. The shapes of these arrays determine the number of observations available for model training.

Table 1 summarizes the dimensions of the datasets generated for the simple linear regression (SLR) and cubic polynomial fitting (CPF) experiments.

Table 1: Array shapes of the generated datasets.

Dataset	**Array Shape**
SLR Data Shape	(120,)
CPF Data Shape	(150,)

The SLR dataset contains 120 observations, while the CPF dataset contains 150 observations. Each dataset is stored as a one-dimensional NumPy array representing the predictor variable values. The corresponding response variables are stored in arrays of the same length.

Because the predictor arrays were stored in one-dimensional form, the regression equations could be implemented directly without constructing a separate design matrix in the reported experiments, which kept the code simpler and easier to interpret.

Using one-dimensional arrays simplifies the implementation of the regression models because vectorized NumPy operations can be applied directly when computing predictions, gradients, and cost functions. This structure also allows the gradient descent algorithms to efficiently process the data during optimization.

Algorithm Implementation

1. Optimization framework

The project uses two gradient-based optimization algorithms: Batch Gradient Descent and Stochastic Gradient Descent. Both methods iteratively update model parameters in the opposite direction of the gradient of the objective function.

In the code, model parameters were stored as NumPy vectors. For simple linear regression, the parameter vector was

$$\boldsymbol{\beta} = [\beta_0, \beta_1]^\top,$$

and for cubic polynomial fitting, the parameter vector was

$$\boldsymbol{\beta} = [\beta_0, \beta_1, \beta_2, \beta_3]^\top.$$

This vector representation makes the updates concise and modular. The same optimizer structure can be reused across different models, provided that the appropriate cost and

gradient functions are supplied.

I used this vector-based structure in the code so that the same update rule could be applied across different regression models by only changing the cost and gradient functions.

2. Batch Gradient Descent pseudocode

Batch Gradient Descent computes the gradient using the full dataset at each update. The pseudocode is given below.

Listing 3: Pseudocode for Batch Gradient Descent.

```
Input: initial beta, learning rate eta, maximum iterations T
for t = 1 to T do
    compute gradient using all observations
    beta <- beta - eta * gradient
    store beta and cost value
end for
return beta history and cost history
```

In my code, this procedure was implemented in the function `batch_gradient_descent()`, which stores both the parameter history and the cost history after each update so that convergence plots could be generated later.

3. Stochastic Gradient Descent pseudocode

Stochastic Gradient Descent updates the parameters using only one observation at a time.

Listing 4: Pseudocode for SGD with replacement.

```
Input: initial beta, learning rate eta, number of epochs E
for epoch = 1 to E do
    for k = 1 to n do
        sample an index i randomly with replacement
        compute gradient using observation i
        beta <- beta - eta * gradient_i
    end for
    store beta and cost value
end for
return beta history and cost history
```

Listing 5: Pseudocode for SGD without replacement.

```
Input: initial beta, learning rate eta, number of epochs E
for epoch = 1 to E do
    shuffle all observation indices
    for each shuffled index i do
        compute gradient using observation i
        beta <- beta - eta * gradient_i
    end for
    store beta and cost value
end for
return beta history and cost history
```

The two SGD variants differ only in how the sample order is generated. In the code, both versions were implemented through the same function `stochastic_gradient_descent()`, with a logical argument indicating whether sampling should occur with or without replacement.

This distinction becomes important in the results section, where SGD without replacement shows slightly more stable convergence, especially for the cubic polynomial model.

Model 1: SLR

1. MLE formulation

For simple linear regression, the fitted value is

$$\hat{y}_i = \beta_0 + \beta_1 x_i.$$

Assuming Gaussian noise,

$$y_i \mid x_i, \beta_0, \beta_1 \sim \mathcal{N}(\beta_0 + \beta_1 x_i, \sigma^2).$$

The likelihood for the dataset is

$$L(\beta_0, \beta_1) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \beta_0 - \beta_1 x_i)^2}{2\sigma^2}\right).$$

Taking the logarithm and dropping constants yields an objective equivalent to minimizing

the sum of squared errors. The cost function used in the code is

$$J_{\text{MLE}}(\beta_0, \beta_1) = \frac{1}{2n} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2.$$

2. Gradient derivation for SLR MLE

Let

$$e_i = y_i - \beta_0 - \beta_1 x_i.$$

Then

$$J_{\text{MLE}}(\beta_0, \beta_1) = \frac{1}{2n} \sum_{i=1}^{n} e_i^2.$$

Differentiating with respect to β_0 gives

$$\frac{\partial J}{\partial \beta_0} = \frac{1}{2n} \sum_{i=1}^{n} 2e_i \, (-1) = -\frac{1}{n} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i).$$

Similarly, differentiating with respect to β_1 gives

$$\frac{\partial J}{\partial \beta_1} = \frac{1}{2n} \sum_{i=1}^{n} 2e_i(-x_i) = -\frac{1}{n} \sum_{i=1}^{n} x_i (y_i - \beta_0 - \beta_1 x_i).$$

These are exactly the quantities computed in the Python gradient function for SLR.

3. SLR parameter estimates under MLE

The code produced the following parameter values for SLR under MLE:

$$\begin{aligned} \text{SLR MLE BGD final beta} &= [2.05870108,\ 2.8057015], \\ \text{SLR MLE SGD WR final beta} &= [2.02758083,\ 2.79455914], \\ \text{SLR MLE SGD WOR final beta} &= [2.0268538,\ 2.73109086]. \end{aligned}$$

These results are close to the true parameters $(2, 3)$, which indicates that all three optimization approaches successfully learned the linear relationship from the noisy data. The BGD estimate is slightly above the true intercept and slightly below the true slope. The two SGD estimates are similar, although the without-replacement version produced a somewhat smaller slope.

The estimates are not exactly equal to the true values because the response variable was generated with Gaussian noise, so the optimization algorithms are fitting a noisy sample

rather than the underlying noiseless relationship.

Figure 3 shows the cost convergence behavior. The most important feature of the plot is that all three methods decrease the objective rapidly and then settle near the optimum.

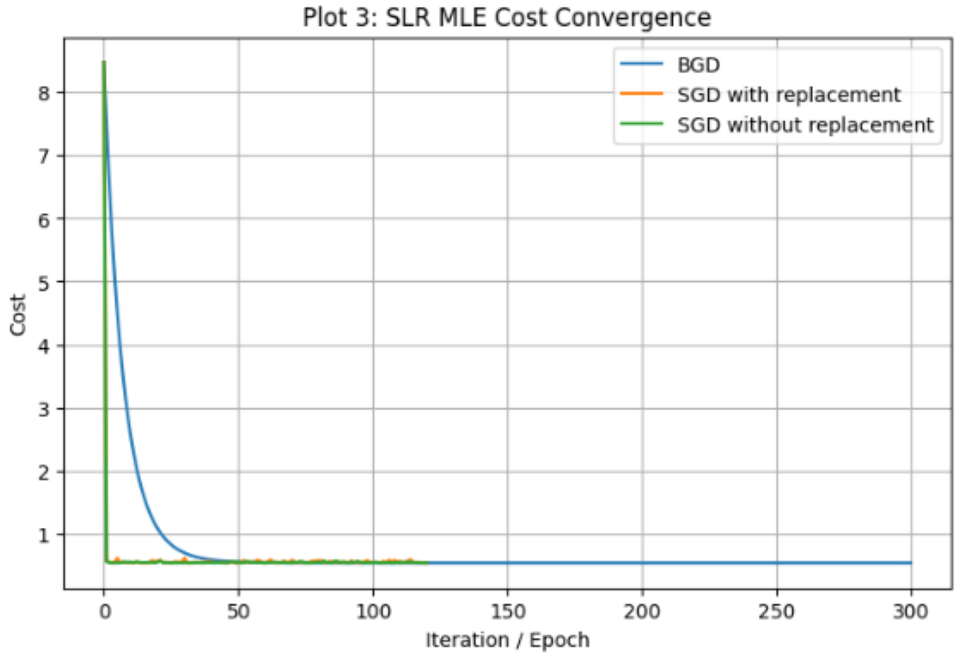

Figure 3: Cost convergence for SLR using Batch Gradient Descent and Stochastic Gradient Descent.

The BGD curve is smooth because each update uses all 120 observations, so the gradient direction is stable. The SGD curves, by contrast, fluctuate slightly because the gradient at each step is computed from only one observation. Nevertheless, the final costs are nearly identical, which shows that all methods successfully minimized the MLE objective.

5. SLR fitted lines

Figure 4 compares the fitted regression lines from all MLE methods against the true line.

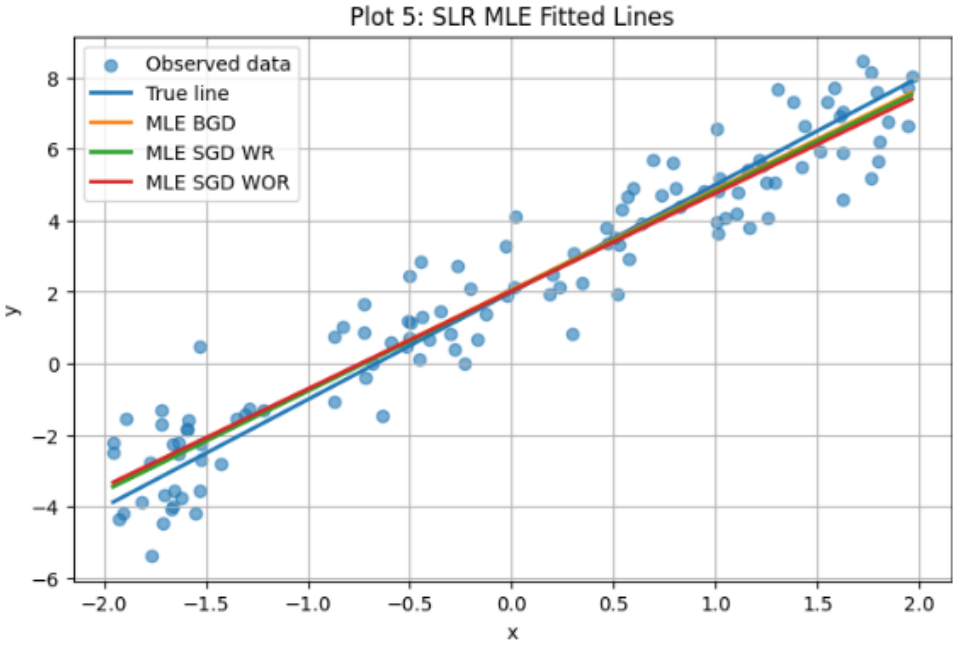

Figure 4: Comparison of fitted regression lines obtained from different optimization methods.

All three fitted lines closely track the data cloud and remain close to the true line. This is an important result because it shows that even though the parameter estimates are not exactly equal to the true values, the predictive behavior of the fitted models is very similar.

This also shows that small differences in parameter estimates do not necessarily lead to large differences in prediction, which is practically important in regression analysis.

6. Cost contour and optimization path

Figure 5 visualizes the MLE cost surface for SLR and overlays the optimization trajectories.

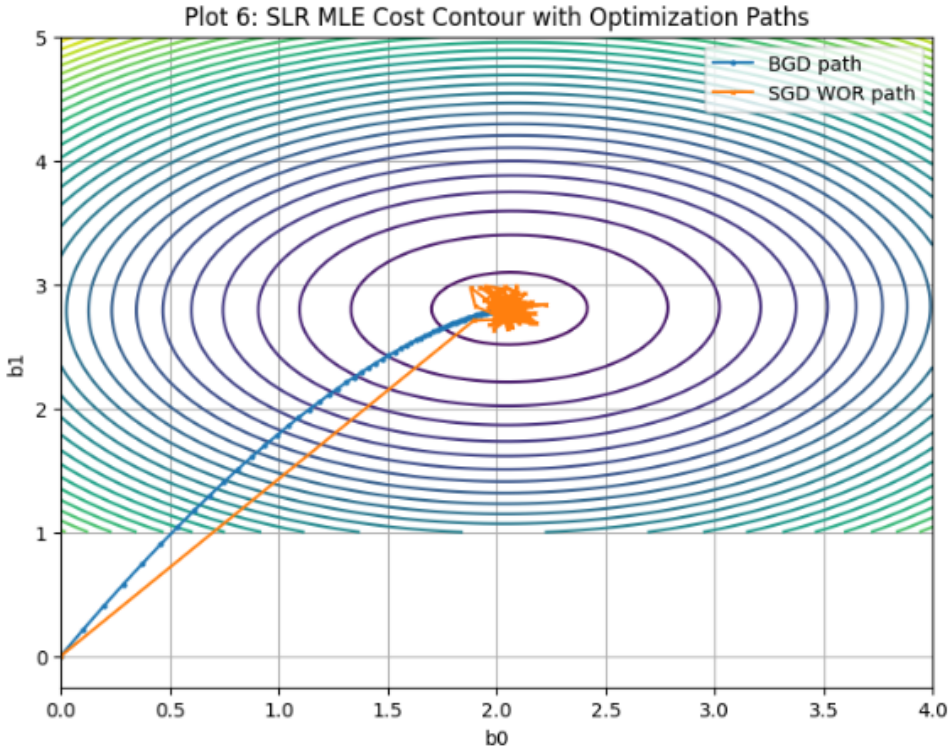

Figure 5: Contour plot of the MLE cost surface with optimization trajectories.

This figure strongly supports the theoretical convexity of the SLR objective. The elliptical level sets indicate that there is a single basin with a unique global minimum. The BGD path follows a direct, smooth route toward the optimum. The SGD without replacement path reaches the same low-cost region but shows small oscillations once it gets close to the minimum, which is exactly what one would expect from noisy single-sample updates.

7. Bayesian formulation for SLR

For the Bayesian model, a Gaussian prior was placed on the slope parameter:

$$\beta_1 \sim \mathcal{N}(\mu_0, \tau^2).$$

In the code, the prior mean was chosen as $\mu_0 = 0$ and the prior variance as $\tau^2 = 2$. The negative log-posterior used for optimization was

$$J_{\text{Bayes}}(\beta_0, \beta_1) = \frac{1}{2\sigma^2} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2 + \frac{1}{2\tau^2} (\beta_1 - \mu_0)^2.$$

The first term corresponds to the data fit, while the second term acts like a shrinkage penalty on the slope.

Differentiating gives

$$\frac{\partial J}{\partial \beta_0} = -\frac{1}{\sigma^2}\sum_{i=1}^{n}(y_i - \beta_0 - \beta_1 x_i),$$

$$\frac{\partial J}{\partial \beta_1} = -\frac{1}{\sigma^2}\sum_{i=1}^{n} x_i(y_i - \beta_0 - \beta_1 x_i) + \frac{1}{\tau^2}(\beta_1 - \mu_0).$$

8. Bayesian SLR results

The code produced the following Bayesian parameter estimates:

$$\text{SLR Bayesian BGD final beta} = [2.05858828,\ 2.79793253],$$
$$\text{SLR Bayesian SGD WR final beta} = [2.0299327,\ 2.80642081],$$
$$\text{SLR Bayesian SGD WOR final beta} = [2.05726878,\ 2.79728237].$$

These values are extremely close to the MLE estimates, but the slopes are slightly pulled toward the prior mean. That is the expected effect of Bayesian shrinkage. Since the fitted Bayesian line is almost identical to the MLE line and the estimated slopes differ only slightly, the observed data appears informative enough that the prior does not dominate the posterior estimate.

Figure 6 shows the convergence of the Bayesian objective.

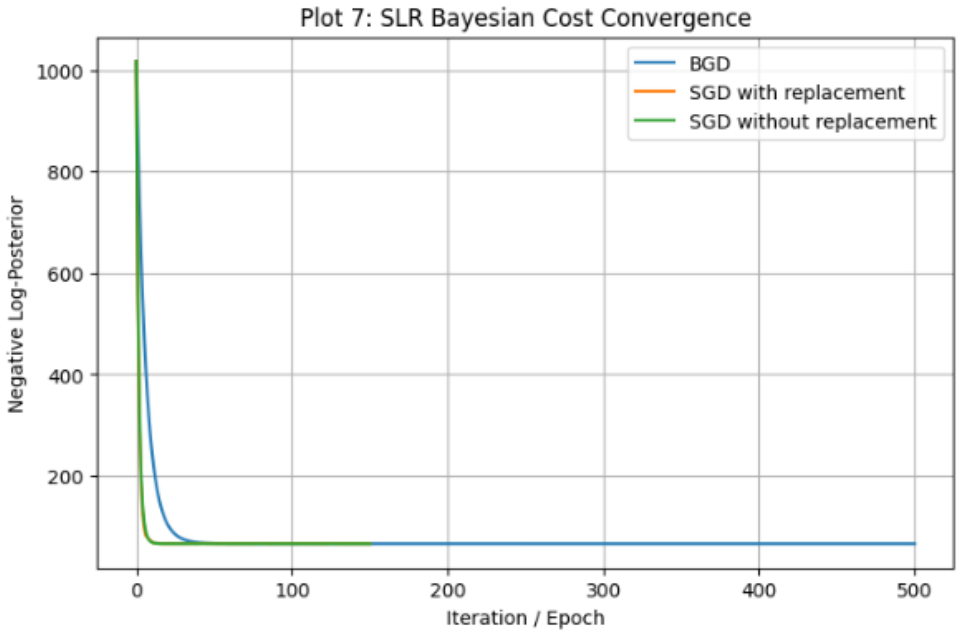

Figure 6: Convergence of the Bayesian log-posterior objective function.

The shape of the convergence plot is similar to the MLE case, but the scale is larger because the objective now includes both the data fit term and the prior penalty term.

Figure 7 compares the fitted line from MLE BGD and Bayesian BGD.

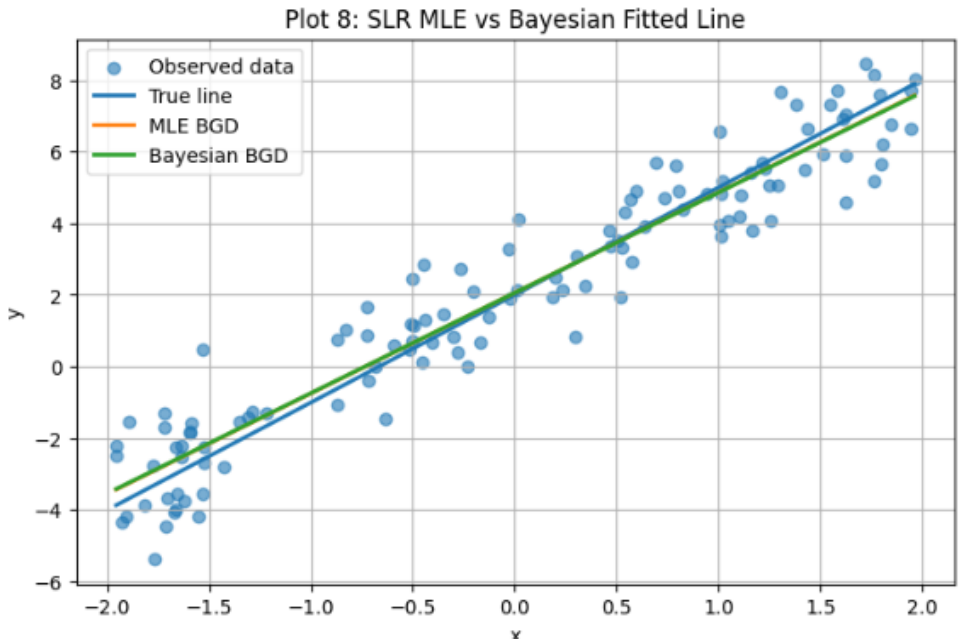

Figure 7: Comparison between MLE and Bayesian regression fits.

The two fitted lines are nearly indistinguishable, which indicates that the observed data were informative enough to dominate the prior. Still, the Bayesian result offers an important conceptual advantage because it incorporates prior information explicitly.

9. SLR final cost comparison

The final cost table from the code reported:

$$\begin{aligned}
\text{SLR MLE BGD} &= 0.535870,\\
\text{SLR MLE SGD WR} &= 0.536442,\\
\text{SLR MLE SGD WOR} &= 0.540519,\\
\text{SLR Bayes BGD} &= 66.266900,\\
\text{SLR Bayes SGD WR} &= 66.323236,\\
\text{SLR Bayes SGD WOR} &= 66.267041.
\end{aligned}$$

The Bayesian cost values are much larger because they correspond to a different objective function. Specifically, they include the prior penalty term in addition to the data-fitting term. Therefore, these values should not be directly compared numerically with the MLE costs. What matters is that all three Bayesian optimizers converged to nearly identical posterior objective values.

Model 2: CPF

1. MLE formulation for cubic polynomial fitting

For cubic polynomial fitting, the prediction function is

$$\hat{y}_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3.$$

Assuming Gaussian noise, the MLE objective again reduces to squared error minimization:

$$J_{\text{CPF}}(\beta_0, \beta_1, \beta_2, \beta_3) = \frac{1}{2n} \sum_{i=1}^{n} \left(y_i - \beta_0 - \beta_1 x_i - \beta_2 x_i^2 - \beta_3 x_i^3 \right)^2.$$

Let

$$e_i = y_i - \beta_0 - \beta_1 x_i - \beta_2 x_i^2 - \beta_3 x_i^3.$$

Then the gradients are

$$\frac{\partial J}{\partial \beta_0} = -\frac{1}{n} \sum_{i=1}^{n} e_i, \qquad \frac{\partial J}{\partial \beta_1} = -\frac{1}{n} \sum_{i=1}^{n} x_i e_i,$$

$$\frac{\partial J}{\partial \beta_2} = -\frac{1}{n} \sum_{i=1}^{n} x_i^2 e_i, \qquad \frac{\partial J}{\partial \beta_3} = -\frac{1}{n} \sum_{i=1}^{n} x_i^3 e_i.$$

These four gradients were implemented directly in the Python function for CPF MLE.

2. True parameters and data generation evidence

The cubic polynomial dataset used in this experiment was generated synthetically in the Python implementation. During the data generation step, the true regression parameters were specified explicitly in the code as

$$\beta_0 = 1.0, \qquad \beta_1 = -2.0, \qquad \beta_2 = 0.5, \qquad \beta_3 = 1.2.$$

These parameters define the underlying cubic function that produces the response variable before noise is added. The dataset was generated using the model

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \varepsilon,$$

where the noise term ε follows a Gaussian distribution with variance $\sigma^2 = 2^2$. After generating the predictor values from a uniform distribution, the response values were

computed directly using these coefficients and the noise term.

The Python console output confirms that the data generation step used the intended parameters. This verification ensures that the optimization algorithms evaluated later in the report are attempting to recover the known coefficients from noisy observations. Because the true coefficients are known, it becomes possible to directly compare the estimated parameters obtained from Batch Gradient Descent and Stochastic Gradient Descent with the ground truth used to generate the dataset.

3. CPF parameter estimates

The code produced the following final parameter estimates:

$$\begin{aligned}
\text{CPF BGD final beta} &= [0.7774746,\ -1.62689787,\ 0.65586954,\ 1.00070916],\\
\text{CPF SGD WR final beta} &= [0.69352684,\ -1.82166981,\ 0.7079886,\ 0.98814378],\\
\text{CPF SGD WOR final beta} &= [0.79445811,\ -1.94778138,\ 0.59816803,\ 1.0010662].
\end{aligned}$$

Relative to the true parameters, all three methods recovered the overall structure of the cubic relationship. The without-replacement SGD estimate came especially close to the true linear coefficient $\beta_1 = -2$. The BGD and SGD estimates differ somewhat in the intercept and the higher-order terms, but the fitted curves remain quite similar.

Although some of the estimated coefficients differ noticeably from the true values, the fitted curves still remain close to the true cubic function, which suggests that multiple nearby coefficient combinations can produce similar predictive behavior in the presence of noise.

4. CPF cost convergence

Figure 8 shows the CPF MLE cost convergence.

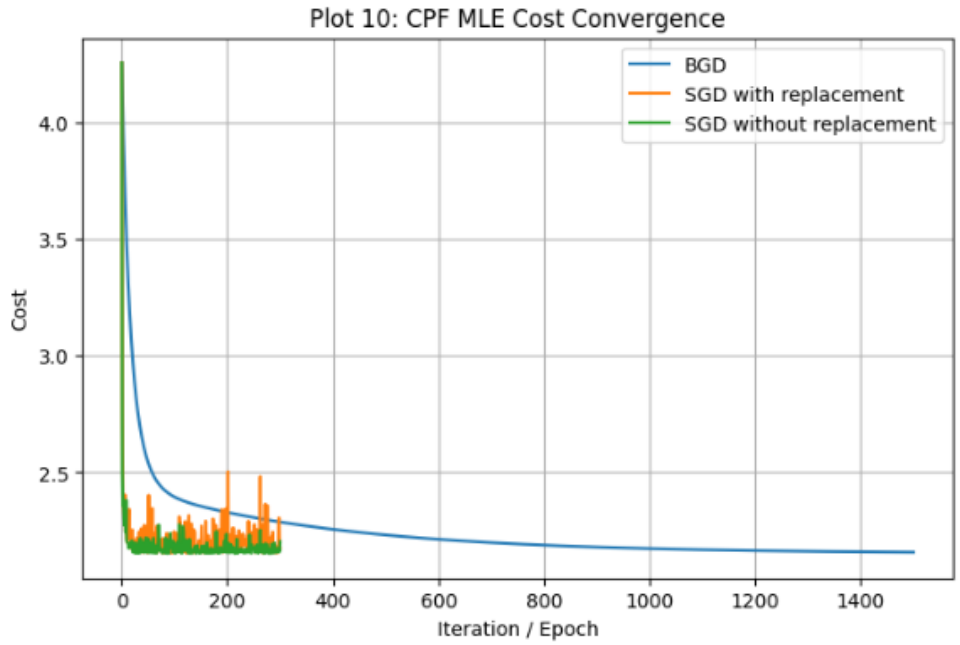

Figure 8: Cost convergence for cubic polynomial fitting under different optimization algorithms.

The cost drops sharply at the beginning of training for all methods. However, compared to the SLR case, the stochastic curves are noticeably noisier. This reflects the greater sensitivity of the polynomial model to sample-level variation, since each update depends on powers of the input variable.

The use of a standardized predictor likely helped reduce unstable updates that could otherwise arise from the squared and cubic terms.

6. CPF fitted curves

Figure 9 compares the fitted curves from BGD and SGD without replacement against the true cubic curve.

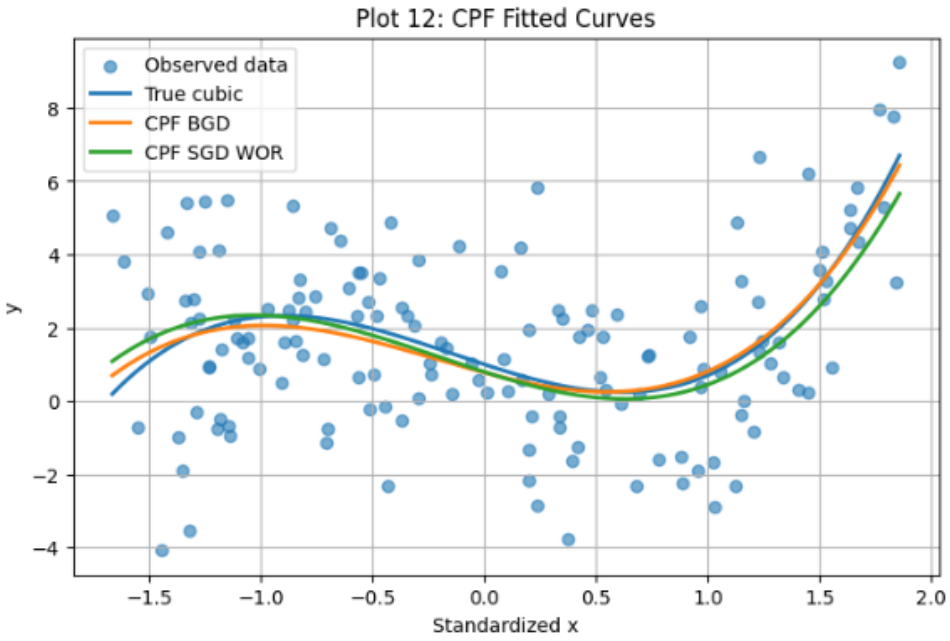

Figure 9: Comparison of fitted cubic curves obtained from gradient descent methods.

The fitted curves follow the true cubic shape closely across most of the predictor range. The deviations that do appear are relatively modest and are consistent with the fact that the observations contain Gaussian noise. This result is important because optimization quality should not be judged only by coefficient error; prediction quality also matters.

7. SGD with and without replacement in CPF

Figure 10 directly compares the two SGD variants for the cubic model.

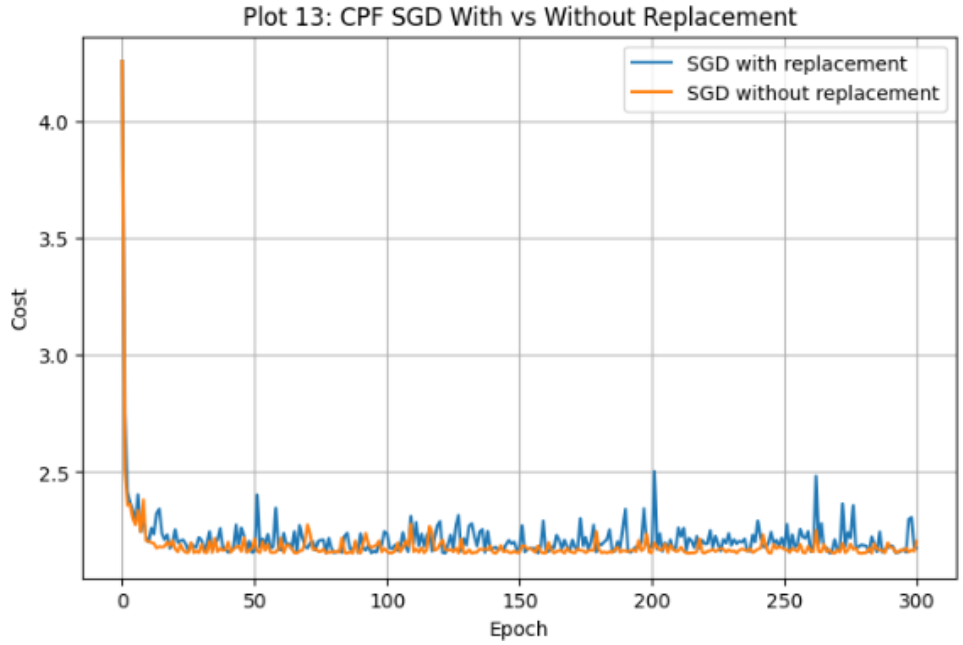

Figure 10: Comparison between SGD with replacement and SGD without replacement.

Both methods quickly reach a low-cost region, but SGD without replacement appears somewhat more stable. This is consistent with the intuition that visiting each sample exactly once per epoch reduces redundant draws and produces a more balanced gradient signal.

8. CPF final cost comparison

The final costs for CPF were reported as

$$\begin{aligned} \text{CPF BGD} &= 2.157663, \\ \text{CPF SGD WR} &= 2.174965, \\ \text{CPF SGD WOR} &= 2.203214. \end{aligned}$$

These values are close to one another, showing that all three methods reached comparable minima. The small differences are reasonable given the noisy data and the stochasticity in SGD.

Overall Results Summary

Instead of presenting screenshots from the Python console, the final outputs are summarized below in LaTeX tables for clarity and readability. These tables report the estimated

parameters and final objective values obtained from the implemented optimization methods.

Parameter Summary

Table 2 presents the final parameter estimates for the simple linear regression model under both MLE and Bayesian formulations. The results are shown for Batch Gradient Descent, SGD with replacement, and SGD without replacement.

Table 2: Final parameter estimates for Simple Linear Regression.

Method	b_0	b_1
True	2.0000	3.0000
SLR MLE BGD	2.0587	2.8057
SLR MLE SGD WR	2.0276	2.7946
SLR MLE SGD WOR	2.0269	2.7311
SLR Bayes BGD	2.0586	2.7979
SLR Bayes SGD WR	2.0299	2.8064
SLR Bayes SGD WOR	2.0573	2.7973

Table 3 presents the final parameter estimates for the cubic polynomial fitting model. Since this model contains four coefficients, the table reports estimates for all parameters under each optimization method.

Table 3: Final parameter estimates for Cubic Polynomial Fitting.

Method	b_0	b_1	b_2	b_3
True	1.0000	-2.0000	0.5000	1.2000
CPF BGD	0.7775	-1.6269	0.6559	1.0007
CPF SGD WR	0.6935	-1.8217	0.7080	0.9881
CPF SGD WOR	0.7945	-1.9478	0.5982	1.0011

The parameter estimates in Tables 2 and 3 show that all optimization methods successfully recovered values close to the true parameters used in the data generation process. For the simple linear regression model, the estimated intercepts and slopes are very similar across all methods, which is consistent with the convex nature of the optimization problem. The Bayesian estimates remain close to the MLE estimates, with slight shrinkage in the slope due to the influence of the prior distribution.

For the cubic polynomial model, the estimates also capture the general structure of the true coefficients. Although the recovered values are not identical to the true parameters, especially for the intercept and linear coefficient, the fitted curves still match the nonlinear pattern of the data well. These deviations are expected because of both the added Gaussian noise and the greater complexity of the polynomial model.

Final Cost Comparison

Table 4 summarizes the final cost values obtained after optimization for each model and algorithm.

Table 4: Final cost comparison across all models and optimization methods.

Model	**Final Cost**
SLR MLE BGD	0.535870
SLR MLE SGD WR	0.536442
SLR MLE SGD WOR	0.540519
SLR Bayes BGD	66.266900
SLR Bayes SGD WR	66.323236
SLR Bayes SGD WOR	66.267041
CPF BGD	2.157663
CPF SGD WR	2.174965
CPF SGD WOR	2.203214

The final cost values confirm that all optimization algorithms converged successfully. For simple linear regression under MLE, the three algorithms achieved nearly identical cost values, with Batch Gradient Descent producing the smallest final cost. This agrees with the earlier convergence plots, where BGD showed smooth and stable progress toward the optimum.

For the Bayesian SLR model, the final objective values are much larger than the MLE values because the Bayesian objective includes an additional penalty term from the prior distribution. Therefore, these values should not be directly compared numerically with the MLE costs. Instead, they should be interpreted within the Bayesian optimization setting, where all three methods again converge to very similar values.

For cubic polynomial fitting, the final costs are slightly higher than in the SLR case, which reflects the increased complexity of the nonlinear model. Among the CPF methods, BGD achieved the lowest final cost, while the SGD variants converged to similar but slightly larger values. This is consistent with the noisier convergence behavior observed in the

CPF cost plots.

Overall, these tables provide a compact summary of the numerical results and confirm the conclusions drawn throughout the report regarding convergence, parameter recovery, and optimization stability.

Discussion

This project investigated how numerical optimization methods estimate regression parameters under different modeling assumptions. Two regression models were examined: simple linear regression (SLR), which represents a convex and low–dimensional optimization problem, and cubic polynomial fitting (CPF), which introduces a higher–dimensional nonlinear regression setting. For both models, parameter estimation was performed using Batch Gradient Descent (BGD) and Stochastic Gradient Descent (SGD), where SGD was implemented with and without replacement. In addition, the SLR model was estimated under both Maximum Likelihood Estimation (MLE) and a Bayesian formulation with a Gaussian prior on the slope parameter.

The numerical results show that all optimization algorithms converged successfully and produced parameter estimates close to the true coefficients used in the synthetic data generation process. In the simple linear regression experiment, the estimated intercept and slope values obtained from BGD and the two SGD variants were very similar, which is consistent with the convex nature of the squared–error objective. Because the SLR model contains only two parameters, the optimization landscape is simple and the algorithms quickly converge to the global minimum. The fitted regression lines produced by different optimization methods are almost indistinguishable and closely follow the true underlying linear relationship.

The cubic polynomial fitting experiment illustrates a more complex optimization problem. The model contains four parameters and includes polynomial terms that interact with each other. As a result, the convergence behavior of the algorithms becomes slightly noisier and the parameter estimates show greater variability compared with the SLR case. Nevertheless, all optimization methods successfully captured the nonlinear structure of the data, and the fitted curves closely matched the true cubic function. These results highlight how increasing model complexity can make optimization more sensitive to stochastic noise and parameter interactions.

A clear difference between BGD and SGD is observed in the convergence behavior. Batch Gradient Descent produces smooth and stable cost curves because each update uses the full dataset to compute the gradient. In contrast, Stochastic Gradient Descent updates parameters using only a single observation at each step, which introduces noise into the

gradient estimates and produces fluctuating optimization paths. Despite this variability, the final parameter estimates obtained by SGD remain very close to those produced by BGD, demonstrating the effectiveness of stochastic optimization methods.

The comparison between SGD with replacement and without replacement shows that the without–replacement variant tends to produce slightly more stable convergence behavior. By ensuring that each observation is used exactly once per epoch, this approach provides a more balanced gradient signal and reduces redundant sampling. This effect is particularly visible in the cubic polynomial model, where the cost curves for SGD without replacement appear smoother.

The Bayesian version of the simple linear regression model demonstrates how prior information can influence parameter estimation. Introducing a Gaussian prior on the slope parameter adds a penalty term to the optimization objective, producing a mild shrinkage effect that pulls the slope estimate slightly toward the prior mean. In this experiment, the difference between the Bayesian and MLE estimates was small because the dataset contained a strong linear signal and the likelihood dominated the posterior distribution.

Overall, the results confirm that gradient-based numerical optimization methods are effective tools for estimating regression parameters in both simple and more complex regression models. Batch Gradient Descent provides stable convergence, while Stochastic Gradient Descent offers a computationally efficient alternative that still achieves comparable solutions. The experiments also illustrate how model complexity, sampling strategy, and prior information can influence optimization behavior and parameter estimates.

Appendix

Appendix A: Full Python Code

```
# ============================================================
# Mini Project 1
# Regression Optimization using Numerical Methods
# Step 1: Imports and Data Generation
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

np.random.seed(42)
```

```
# ------------------------------------------------------------
# Generate data for Simple Linear Regression (SLR)
# y = b0 + b1*x + noise
# ------------------------------------------------------------
n_slr = 120
b0_true_slr = 2.0
b1_true_slr = 3.0
sigma_slr = 1.0

x_slr = np.random.uniform(-2, 2, n_slr)
eps_slr = np.random.normal(0, sigma_slr, n_slr)
y_slr = b0_true_slr + b1_true_slr * x_slr + eps_slr

# ------------------------------------------------------------
# Generate data for Cubic Polynomial Fitting (CPF)
# y = b0 + b1*x + b2*x^2 + b3*x^3 + noise
# ------------------------------------------------------------
n_cpf = 150
b0_true_cpf = 1.0
b1_true_cpf = -2.0
b2_true_cpf = 0.5# ============================================================
# Mini Project 1
# Regression Optimization using Numerical Methods
# Step 1: Imports and setup
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

np.random.seed(42)

# nicer plot size
plt.rcParams["figure.figsize"] = (8, 5)
plt.rcParams["axes.grid"] = True
b3_true_cpf = 1.2
sigma_cpf = 2.0
```

```
x_cpf_raw = np.random.uniform(-2, 2, n_cpf)

# standardize x for numerical stability
x_cpf_mean = np.mean(x_cpf_raw)
x_cpf_std = np.std(x_cpf_raw)
x_cpf = (x_cpf_raw - x_cpf_mean) / x_cpf_std

eps_cpf = np.random.normal(0, sigma_cpf, n_cpf)
y_cpf = (
    b0_true_cpf
    + b1_true_cpf * x_cpf
    + b2_true_cpf * x_cpf**2
    + b3_true_cpf * x_cpf**3
    + eps_cpf
)

print("SLR data shape:", x_slr.shape, y_slr.shape)
print("CPF data shape:", x_cpf.shape, y_cpf.shape)

# ============================================================
# Step 2: Generate data for Simple Linear Regression (SLR)
# y = b0 + b1*x + noise
# ============================================================

n_slr = 120
b0_true_slr = 2.0
b1_true_slr = 3.0
sigma_slr = 1.0

x_slr = np.random.uniform(-2, 2, n_slr)
eps_slr = np.random.normal(0, sigma_slr, n_slr)
y_slr = b0_true_slr + b1_true_slr * x_slr + eps_slr

print("SLR data generated.")
print("True parameters:", b0_true_slr, b1_true_slr)

# ============================================================
# Step 3: Plot SLR generated data
# ============================================================
```

```
plt.figure()
plt.scatter(x_slr, y_slr, alpha=0.7, label="Observed data")

x_line = np.linspace(x_slr.min(), x_slr.max(), 300)
y_line_true = b0_true_slr + b1_true_slr * x_line
plt.plot(x_line, y_line_true, linewidth=2, label="True line")

plt.title("Plot 1: SLR Generated Data")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()

# ============================================================
# Step 4: Batch Gradient Descent (BGD)
# ============================================================
# Pseudocode:
#
# Input: learning rate eta, initial theta, max_iters
# For t in 1,...,max_iters:
#     Compute full gradient using all training data
#     theta = theta - eta * gradient
#     Store theta and cost
# Return theta, history
# ============================================================

def batch_gradient_descent(cost_fn, grad_fn, beta_init, x, y, lr=0.01, n_iters=500, *
    beta = beta_init.copy()
    beta_history = [beta.copy()]
    cost_history = [cost_fn(beta, x, y, **kwargs)]

    for _ in range(n_iters):
        grad = grad_fn(beta, x, y, **kwargs)
        beta = beta - lr * grad

        beta_history.append(beta.copy())
        cost_history.append(cost_fn(beta, x, y, **kwargs))
```

```
    return np.array(beta_history), np.array(cost_history)
# ============================================================
# Step 4: Generate data for Cubic Polynomial Fitting (CPF)
# y = b0 + b1*x + b2*x^2 + b3*x^3 + noise
# ============================================================

n_cpf = 150
b0_true_cpf = 1.0
b1_true_cpf = -2.0
b2_true_cpf = 0.5
b3_true_cpf = 1.2
sigma_cpf = 2.0

x_cpf_raw = np.random.uniform(-2, 2, n_cpf)

# standardize x for numerical stability
x_cpf_mean = np.mean(x_cpf_raw)
x_cpf_std = np.std(x_cpf_raw)
x_cpf = (x_cpf_raw - x_cpf_mean) / x_cpf_std

eps_cpf = np.random.normal(0, sigma_cpf, n_cpf)
y_cpf = (
    b0_true_cpf
    + b1_true_cpf * x_cpf
    + b2_true_cpf * x_cpf**2
    + b3_true_cpf * x_cpf**3
    + eps_cpf
)

print("CPF data generated.")
print("True parameters:", b0_true_cpf, b1_true_cpf, b2_true_cpf, b3_true_cpf)

# ============================================================
# Step 5: Plot CPF generated data
# ============================================================

plt.figure()
plt.scatter(x_cpf, y_cpf, alpha=0.7, label="Observed data")
```

```
x_curve = np.linspace(x_cpf.min(), x_cpf.max(), 400)
y_curve_true = (
    b0_true_cpf
    + b1_true_cpf * x_curve
    + b2_true_cpf * x_curve**2
    + b3_true_cpf * x_curve**3
)
plt.plot(x_curve, y_curve_true, linewidth=2, label="True cubic curve")

plt.title("Plot 2: CPF Generated Data")
plt.xlabel("Standardized x")
plt.ylabel("y")
plt.legend()
plt.show()
# ============================================================
# Step 6: Prediction functions
# ============================================================

def slr_predict(x, beta):
    # beta = [b0, b1]
    return beta[0] + beta[1] * x


def cpf_predict(x, beta):
    # beta = [b0, b1, b2, b3]
    return beta[0] + beta[1]*x + beta[2]*x**2 + beta[3]*x**3
# ============================================================
# Step 7: Cost functions
# ============================================================

def slr_mle_cost(beta, x, y):
    y_hat = slr_predict(x, beta)
    return 0.5 * np.mean((y - y_hat)**2)


def slr_bayes_cost(beta, x, y, sigma2=1.0, mu_prior=0.0, tau2=2.0, **kwargs):
    """
    Negative log-posterior up to a constant:
    J = (1/(2*sigma2)) sum (y-yhat)^2 + (1/(2*tau2))(b1-mu_prior)^2
```

```
    """
    y_hat = slr_predict(x, beta)
    data_term = (1 / (2 * sigma2)) * np.sum((y - y_hat)**2)
    prior_term = (1 / (2 * tau2)) * (beta[1] - mu_prior)**2
    return data_term + prior_term


def cpf_mle_cost(beta, x, y):
    y_hat = cpf_predict(x, beta)
    return 0.5 * np.mean((y - y_hat)**2)
# ============================================================
# Step 8: Full-batch gradients
# ============================================================

def slr_mle_grad(beta, x, y):
    y_hat = slr_predict(x, beta)
    error = y - y_hat
    db0 = -np.mean(error)
    db1 = -np.mean(x * error)
    return np.array([db0, db1])


def slr_bayes_grad(beta, x, y, sigma2=1.0, mu_prior=0.0, tau2=2.0):
    y_hat = slr_predict(x, beta)
    error = y - y_hat
    db0 = -(1 / sigma2) * np.sum(error)
    db1 = -(1 / sigma2) * np.sum(x * error) + (1 / tau2) * (beta[1] - mu_prior)
    return np.array([db0, db1])


def cpf_mle_grad(beta, x, y):
    y_hat = cpf_predict(x, beta)
    error = y - y_hat

    db0 = -np.mean(error)
    db1 = -np.mean(x * error)
    db2 = -np.mean((x**2) * error)
    db3 = -np.mean((x**3) * error)
```

```
    return np.array([db0, db1, db2, db3])
# ============================================================
# Step 9: Single-sample gradients for SGD
# ============================================================

def slr_mle_grad_single(beta, x_i, y_i):
    error = y_i - (beta[0] + beta[1] * x_i)
    db0 = -error
    db1 = -x_i * error
    return np.array([db0, db1])


def slr_bayes_grad_single(beta, x_i, y_i, n, sigma2=1.0, mu_prior=0.0, tau2=2.0):
    error = y_i - (beta[0] + beta[1] * x_i)
    db0 = -(1 / sigma2) * error
    db1 = -(x_i / sigma2) * error + (1 / (n * tau2)) * (beta[1] - mu_prior)
    return np.array([db0, db1])


def cpf_mle_grad_single(beta, x_i, y_i):
    y_hat = beta[0] + beta[1]*x_i + beta[2]*x_i**2 + beta[3]*x_i**3
    error = y_i - y_hat

    db0 = -error
    db1 = -x_i * error
    db2 = -(x_i**2) * error
    db3 = -(x_i**3) * error

    return np.array([db0, db1, db2, db3])
# ============================================================
# Step 10: Batch Gradient Descent (BGD)
# ============================================================

def batch_gradient_descent(cost_fn, grad_fn, beta_init, x, y, lr=0.01, n_iters=500, *
    beta = beta_init.copy()
    beta_history = [beta.copy()]
    cost_history = [cost_fn(beta, x, y, **kwargs)]

    for _ in range(n_iters):
```

```
        grad = grad_fn(beta, x, y, **kwargs)
        beta = beta - lr * grad

        beta_history.append(beta.copy())
        cost_history.append(cost_fn(beta, x, y, **kwargs))

    return np.array(beta_history), np.array(cost_history)
# ============================================================
# Step 11: Corrected Stochastic Gradient Descent (SGD)
# ============================================================

def stochastic_gradient_descent(
    beta_init,
    x,
    y,
    single_grad_fn,
    full_cost_fn,
    lr=0.01,
    epochs=100,
    with_replacement=True,
    **kwargs
):
    beta = beta_init.copy()
    n = len(x)

    # kwargs for cost and gradient handled separately
    cost_kwargs = kwargs.copy()
    grad_kwargs = kwargs.copy()

    # cost function does not need n
    if "n" in cost_kwargs:
        cost_kwargs.pop("n")

    beta_history = [beta.copy()]
    cost_history = [full_cost_fn(beta, x, y, **cost_kwargs)]

    for _ in range(epochs):
        if with_replacement:
            for _ in range(n):
```

```
                i = np.random.randint(0, n)
                grad = single_grad_fn(beta, x[i], y[i], **grad_kwargs)
                beta = beta - lr * grad
        else:
            indices = np.random.permutation(n)
            for i in indices:
                grad = single_grad_fn(beta, x[i], y[i], **grad_kwargs)
                beta = beta - lr * grad

        beta_history.append(beta.copy())
        cost_history.append(full_cost_fn(beta, x, y, **cost_kwargs))

    return np.array(beta_history), np.array(cost_history)
# ============================================================
# Step 12: Run SLR with MLE
# ============================================================

beta_init_slr = np.array([0.0, 0.0])

# BGD
slr_mle_bgd_beta_hist, slr_mle_bgd_cost_hist = batch_gradient_descent(
    cost_fn=slr_mle_cost,
    grad_fn=slr_mle_grad,
    beta_init=beta_init_slr,
    x=x_slr,
    y=y_slr,
    lr=0.05,
    n_iters=300
)

# SGD with replacement
slr_mle_sgd_wr_beta_hist, slr_mle_sgd_wr_cost_hist = stochastic_gradient_descent(
    beta_init=beta_init_slr,
    x=x_slr,
    y=y_slr,
    single_grad_fn=slr_mle_grad_single,
    full_cost_fn=slr_mle_cost,
    lr=0.02,
    epochs=120,
```

```
    with_replacement=True
)

# SGD without replacement
slr_mle_sgd_wor_beta_hist, slr_mle_sgd_wor_cost_hist = stochastic_gradient_descent(
    beta_init=beta_init_slr,
    x=x_slr,
    y=y_slr,
    single_grad_fn=slr_mle_grad_single,
    full_cost_fn=slr_mle_cost,
    lr=0.02,
    epochs=120,
    with_replacement=False
)

print("SLR MLE BGD final beta:", slr_mle_bgd_beta_hist[-1])
print("SLR MLE SGD WR final beta:", slr_mle_sgd_wr_beta_hist[-1])
print("SLR MLE SGD WOR final beta:", slr_mle_sgd_wor_beta_hist[-1])
# ============================================================
# Step 13: Plot 3 - SLR MLE cost convergence
# ============================================================

plt.figure()
plt.plot(slr_mle_bgd_cost_hist, label="BGD")
plt.plot(slr_mle_sgd_wr_cost_hist, label="SGD with replacement")
plt.plot(slr_mle_sgd_wor_cost_hist, label="SGD without replacement")
plt.title("Plot 3: SLR MLE Cost Convergence")
plt.xlabel("Iteration / Epoch")
plt.ylabel("Cost")
plt.legend()
plt.show()
# ============================================================
# Step 14: Plot 4 - SLR MLE parameter convergence
# ============================================================

plt.figure()
plt.plot(slr_mle_bgd_beta_hist[:, 0], label="b0 (BGD)")
plt.plot(slr_mle_bgd_beta_hist[:, 1], label="b1 (BGD)")
plt.title("Plot 4: SLR MLE Parameter Convergence (BGD)")
```

```
plt.xlabel("Iteration")
plt.ylabel("Parameter value")
plt.legend()
plt.show()
# ============================================================
# Step 15: Plot 5 - SLR fitted lines
# ============================================================

plt.figure()
plt.scatter(x_slr, y_slr, alpha=0.6, label="Observed data")

x_line = np.linspace(x_slr.min(), x_slr.max(), 300)

plt.plot(x_line, b0_true_slr + b1_true_slr * x_line, linewidth=2, label="True line")

beta_bgd = slr_mle_bgd_beta_hist[-1]
plt.plot(x_line, beta_bgd[0] + beta_bgd[1] * x_line, linewidth=2, label="MLE BGD")

beta_sgd_wr = slr_mle_sgd_wr_beta_hist[-1]
plt.plot(x_line, beta_sgd_wr[0] + beta_sgd_wr[1] * x_line, linewidth=2, label="MLE SG

beta_sgd_wor = slr_mle_sgd_wor_beta_hist[-1]
plt.plot(x_line, beta_sgd_wor[0] + beta_sgd_wor[1] * x_line, linewidth=2, label="MLE

plt.title("Plot 5: SLR MLE Fitted Lines")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
# ============================================================
# Step 16: Plot 6 - SLR MLE contour with optimization paths
# ============================================================

b0_vals = np.linspace(0, 4, 100)
b1_vals = np.linspace(1, 5, 100)

B0, B1 = np.meshgrid(b0_vals, b1_vals)
Z = np.zeros_like(B0)
```

```
for i in range(B0.shape[0]):
    for j in range(B0.shape[1]):
        beta_temp = np.array([B0[i, j], B1[i, j]])
        Z[i, j] = slr_mle_cost(beta_temp, x_slr, y_slr)

plt.figure(figsize=(8, 6))
plt.contour(B0, B1, Z, levels=30)

plt.plot(
    slr_mle_bgd_beta_hist[:, 0],
    slr_mle_bgd_beta_hist[:, 1],
    marker='o',
    markersize=2,
    label='BGD path'
)

plt.plot(
    slr_mle_sgd_wor_beta_hist[:, 0],
    slr_mle_sgd_wor_beta_hist[:, 1],
    marker='x',
    markersize=2,
    label='SGD WOR path'
)

plt.title("Plot 6: SLR MLE Cost Contour with Optimization Paths")
plt.xlabel("b0")
plt.ylabel("b1")
plt.legend()
plt.show()
# ============================================================
# Step 17: Run SLR with Bayesian objective
# ============================================================

mu_prior = 0.0
tau2 = 2.0
sigma2 = sigma_slr**2

beta_init_slr_bayes = np.array([0.0, 0.0])
```

```
# BGD
slr_bayes_bgd_beta_hist, slr_bayes_bgd_cost_hist = batch_gradient_descent(
    cost_fn=slr_bayes_cost,
    grad_fn=slr_bayes_grad,
    beta_init=beta_init_slr_bayes,
    x=x_slr,
    y=y_slr,
    lr=0.0005,
    n_iters=500,
    sigma2=sigma2,
    mu_prior=mu_prior,
    tau2=tau2
)

# SGD with replacement
slr_bayes_sgd_wr_beta_hist, slr_bayes_sgd_wr_cost_hist = stochastic_gradient_descent(
    beta_init=beta_init_slr_bayes,
    x=x_slr,
    y=y_slr,
    single_grad_fn=slr_bayes_grad_single,
    full_cost_fn=slr_bayes_cost,
    lr=0.002,
    epochs=150,
    with_replacement=True,
    n=len(x_slr),
    sigma2=sigma2,
    mu_prior=mu_prior,
    tau2=tau2
)

# SGD without replacement
slr_bayes_sgd_wor_beta_hist, slr_bayes_sgd_wor_cost_hist = stochastic_gradient_descen
    beta_init=beta_init_slr_bayes,
    x=x_slr,
    y=y_slr,
    single_grad_fn=slr_bayes_grad_single,
    full_cost_fn=slr_bayes_cost,
    lr=0.002,
    epochs=150,
```

```
    with_replacement=False,
    n=len(x_slr),
    sigma2=sigma2,
    mu_prior=mu_prior,
    tau2=tau2
)

print("SLR Bayesian BGD final beta:", slr_bayes_bgd_beta_hist[-1])
print("SLR Bayesian SGD WR final beta:", slr_bayes_sgd_wr_beta_hist[-1])
print("SLR Bayesian SGD WOR final beta:", slr_bayes_sgd_wor_beta_hist[-1])
# ============================================================
# Step 18: Plot 7 - SLR Bayesian cost convergence
# ============================================================

plt.figure()
plt.plot(slr_bayes_bgd_cost_hist, label="BGD")
plt.plot(slr_bayes_sgd_wr_cost_hist, label="SGD with replacement")
plt.plot(slr_bayes_sgd_wor_cost_hist, label="SGD without replacement")
plt.title("Plot 7: SLR Bayesian Cost Convergence")
plt.xlabel("Iteration / Epoch")
plt.ylabel("Negative Log-Posterior")
plt.legend()
plt.show()
# ============================================================
# Step 19: Plot 8 - SLR MLE vs Bayesian fitted line
# ============================================================

plt.figure()
plt.scatter(x_slr, y_slr, alpha=0.6, label="Observed data")

x_line = np.linspace(x_slr.min(), x_slr.max(), 300)

plt.plot(
    x_line,
    b0_true_slr + b1_true_slr * x_line,
    linewidth=2,
    label="True line"
)
```

```
beta_mle = slr_mle_bgd_beta_hist[-1]
plt.plot(
    x_line,
    beta_mle[0] + beta_mle[1] * x_line,
    linewidth=2,
    label="MLE BGD"
)

beta_bayes = slr_bayes_bgd_beta_hist[-1]
plt.plot(
    x_line,
    beta_bayes[0] + beta_bayes[1] * x_line,
    linewidth=2,
    label="Bayesian BGD"
)

plt.title("Plot 8: SLR MLE vs Bayesian Fitted Line")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
# ============================================================
# Step 20: Plot 9 - Bayesian shrinkage on slope
# ============================================================

labels = ["True slope", "MLE slope", "Bayesian slope"]
values = [
    b1_true_slr,
    slr_mle_bgd_beta_hist[-1][1],
    slr_bayes_bgd_beta_hist[-1][1]
]

plt.figure()
plt.bar(labels, values)
plt.title("Plot 9: Comparison of Slope Estimates")
plt.ylabel("Slope value")
plt.show()
# ============================================================
# Step 21: Run CPF with MLE
```

```
# ============================================================

beta_init_cpf = np.array([0.0, 0.0, 0.0, 0.0])

# BGD
cpf_bgd_beta_hist, cpf_bgd_cost_hist = batch_gradient_descent(
    cost_fn=cpf_mle_cost,
    grad_fn=cpf_mle_grad,
    beta_init=beta_init_cpf,
    x=x_cpf,
    y=y_cpf,
    lr=0.01,
    n_iters=1500
)

# SGD with replacement
cpf_sgd_wr_beta_hist, cpf_sgd_wr_cost_hist = stochastic_gradient_descent(
    beta_init=beta_init_cpf,
    x=x_cpf,
    y=y_cpf,
    single_grad_fn=cpf_mle_grad_single,
    full_cost_fn=cpf_mle_cost,
    lr=0.005,
    epochs=300,
    with_replacement=True
)

# SGD without replacement
cpf_sgd_wor_beta_hist, cpf_sgd_wor_cost_hist = stochastic_gradient_descent(
    beta_init=beta_init_cpf,
    x=x_cpf,
    y=y_cpf,
    single_grad_fn=cpf_mle_grad_single,
    full_cost_fn=cpf_mle_cost,
    lr=0.005,
    epochs=300,
    with_replacement=False
)
```

```
print("CPF BGD final beta:", cpf_bgd_beta_hist[-1])
print("CPF SGD WR final beta:", cpf_sgd_wr_beta_hist[-1])
print("CPF SGD WOR final beta:", cpf_sgd_wor_beta_hist[-1])
# ============================================================
# Step 22: Plot 10 - CPF cost convergence
# ============================================================

plt.figure()
plt.plot(cpf_bgd_cost_hist, label="BGD")
plt.plot(cpf_sgd_wr_cost_hist, label="SGD with replacement")
plt.plot(cpf_sgd_wor_cost_hist, label="SGD without replacement")
plt.title("Plot 10: CPF MLE Cost Convergence")
plt.xlabel("Iteration / Epoch")
plt.ylabel("Cost")
plt.legend()
plt.show()
# ============================================================
# Step 23: Plot 11 - CPF parameter convergence
# ============================================================

plt.figure(figsize=(10, 6))
plt.plot(cpf_bgd_beta_hist[:, 0], label="b0")
plt.plot(cpf_bgd_beta_hist[:, 1], label="b1")
plt.plot(cpf_bgd_beta_hist[:, 2], label="b2")
plt.plot(cpf_bgd_beta_hist[:, 3], label="b3")
plt.title("Plot 11: CPF Parameter Convergence (BGD)")
plt.xlabel("Iteration")
plt.ylabel("Coefficient value")
plt.legend()
plt.show()
# ============================================================
# Step 24: Plot 12 - CPF fitted curves
# ============================================================

plt.figure()
plt.scatter(x_cpf, y_cpf, alpha=0.6, label="Observed data")

x_curve = np.linspace(x_cpf.min(), x_cpf.max(), 400)
```

```
plt.plot(
    x_curve,
    b0_true_cpf + b1_true_cpf*x_curve + b2_true_cpf*x_curve**2 + b3_true_cpf*x_curve**
    linewidth=2,
    label="True cubic"
)

beta_cpf_bgd = cpf_bgd_beta_hist[-1]
plt.plot(
    x_curve,
    cpf_predict(x_curve, beta_cpf_bgd),
    linewidth=2,
    label="CPF BGD"
)

beta_cpf_sgd_wor = cpf_sgd_wor_beta_hist[-1]
plt.plot(
    x_curve,
    cpf_predict(x_curve, beta_cpf_sgd_wor),
    linewidth=2,
    label="CPF SGD WOR"
)

plt.title("Plot 12: CPF Fitted Curves")
plt.xlabel("Standardized x")
plt.ylabel("y")
plt.legend()
plt.show()
# ============================================================
# Step 25: Plot 13 - CPF SGD comparison
# ============================================================

plt.figure()
plt.plot(cpf_sgd_wr_cost_hist, label="SGD with replacement")
plt.plot(cpf_sgd_wor_cost_hist, label="SGD without replacement")
plt.title("Plot 13: CPF SGD With vs Without Replacement")
plt.xlabel("Epoch")
plt.ylabel("Cost")
plt.legend()
```

```
plt.show()
# ============================================================
# Step 26: Summary tables
# ============================================================

slr_results = pd.DataFrame({
    "Method": [
        "True",
        "SLR MLE BGD",
        "SLR MLE SGD WR",
        "SLR MLE SGD WOR",
        "SLR Bayes BGD",
        "SLR Bayes SGD WR",
        "SLR Bayes SGD WOR"
    ],
    "b0": [
        b0_true_slr,
        slr_mle_bgd_beta_hist[-1][0],
        slr_mle_sgd_wr_beta_hist[-1][0],
        slr_mle_sgd_wor_beta_hist[-1][0],
        slr_bayes_bgd_beta_hist[-1][0],
        slr_bayes_sgd_wr_beta_hist[-1][0],
        slr_bayes_sgd_wor_beta_hist[-1][0],
    ],
    "b1": [
        b1_true_slr,
        slr_mle_bgd_beta_hist[-1][1],
        slr_mle_sgd_wr_beta_hist[-1][1],
        slr_mle_sgd_wor_beta_hist[-1][1],
        slr_bayes_bgd_beta_hist[-1][1],
        slr_bayes_sgd_wr_beta_hist[-1][1],
        slr_bayes_sgd_wor_beta_hist[-1][1],
    ]
})

print("\nSLR Results Summary")
print(slr_results.round(4))
```

```
cpf_results = pd.DataFrame({
    "Method": [
        "True",
        "CPF BGD",
        "CPF SGD WR",
        "CPF SGD WOR"
    ],
    "b0": [
        b0_true_cpf,
        cpf_bgd_beta_hist[-1][0],
        cpf_sgd_wr_beta_hist[-1][0],
        cpf_sgd_wor_beta_hist[-1][0]
    ],
    "b1": [
        b1_true_cpf,
        cpf_bgd_beta_hist[-1][1],
        cpf_sgd_wr_beta_hist[-1][1],
        cpf_sgd_wor_beta_hist[-1][1]
    ],
    "b2": [
        b2_true_cpf,
        cpf_bgd_beta_hist[-1][2],
        cpf_sgd_wr_beta_hist[-1][2],
        cpf_sgd_wor_beta_hist[-1][2]
    ],
    "b3": [
        b3_true_cpf,
        cpf_bgd_beta_hist[-1][3],
        cpf_sgd_wr_beta_hist[-1][3],
        cpf_sgd_wor_beta_hist[-1][3]
    ]
})

print("\nCPF Results Summary")
print(cpf_results.round(4))
# ============================================================
# Step 27: Final cost comparison table
# ============================================================
```

```
performance_table = pd.DataFrame({
    "Model": [
        "SLR MLE BGD",
        "SLR MLE SGD WR",
        "SLR MLE SGD WOR",
        "SLR Bayes BGD",
        "SLR Bayes SGD WR",
        "SLR Bayes SGD WOR",
        "CPF BGD",
        "CPF SGD WR",
        "CPF SGD WOR"
    ],
    "Final Cost": [
        slr_mle_bgd_cost_hist[-1],
        slr_mle_sgd_wr_cost_hist[-1],
        slr_mle_sgd_wor_cost_hist[-1],
        slr_bayes_bgd_cost_hist[-1],
        slr_bayes_sgd_wr_cost_hist[-1],
        slr_bayes_sgd_wor_cost_hist[-1],
        cpf_bgd_cost_hist[-1],
        cpf_sgd_wr_cost_hist[-1],
        cpf_sgd_wor_cost_hist[-1]
    ]
})

print("\nFinal Cost Comparison")
print(performance_table.round(6))
```